Documentazione OO - UninaFoodLab

Giuseppe Curi N86005091 - Ludovica I. Cafasso N86005226

2024-2025

Docente: S. Di Martino

Indice

1 Progettazione **3**
1.1 Implementazione della suddivisione MVC 3
1.2 Model 4
1.2.1 Descrizione delle entità: 5
1.3 DAO 6
1.3.1 Riassunto e classi DAO 7
1.4 Controller 8
1.4.1 Riassunto e classi Controller 9
1.5 View 10
1.6 GitHub 15

Breve descrizione del dominio

UninaFoodLab è un sistema che permette la gestione, da parte di docenti-chef dotati di account personale, di un insieme di corsi fruibili da studenti interessati alle varie lezioni culinarie proposte in due metodologie precise: presenza o online, entrambe predisposte ad essere legate a corsi precedentemente creati; ogni sessione pratica può includere la preparazione di ricette diverse, scelte dallo chef sulla base del corso di cui saranno protagoniste.

Gli chef possono interagire con gli studenti attraverso messaggi inviati sottoforma di notifiche. Ogni mese è possibile consultare un report, i cui valori sono salvabili sul proprio pc, contenente le principali informazioni sulle sessioni svolte. La gestione e visualizzazione dei vari elementi è attuabile attraverso un'interfaccia Swing.

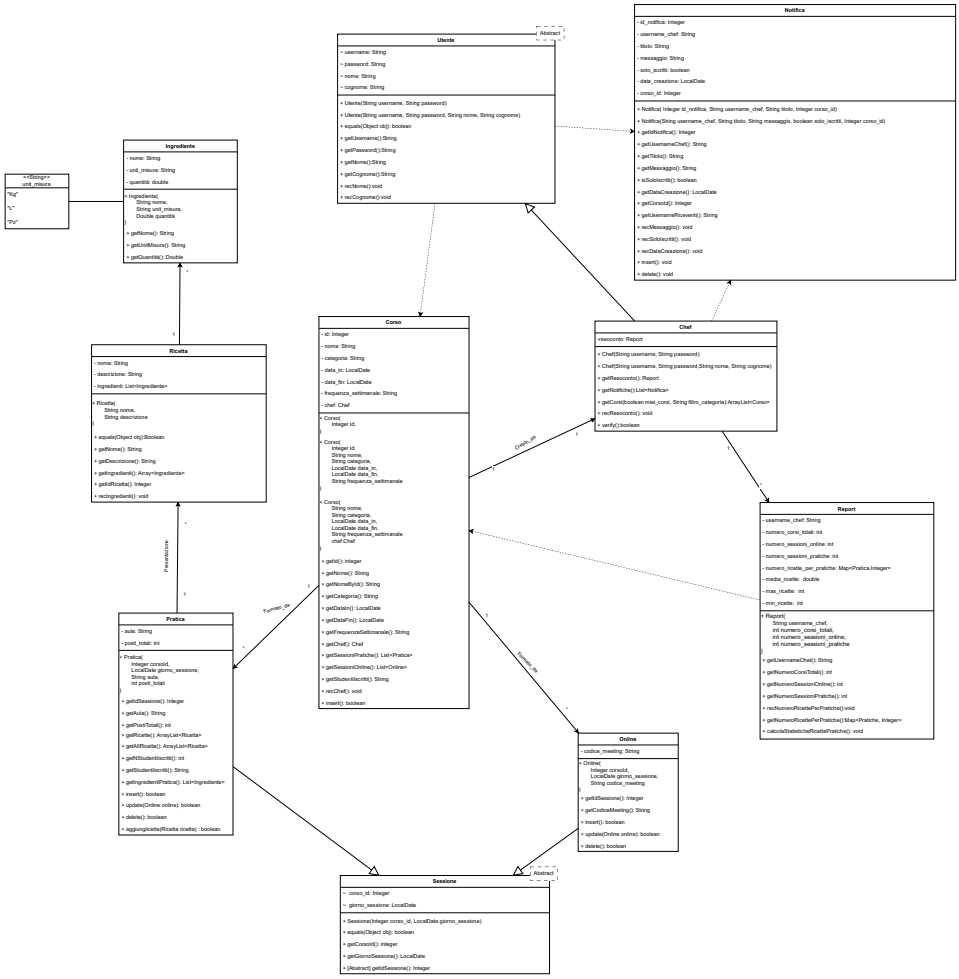
1 Progettazione

1.1 Implementazione della suddivisione MVC

L'applicazione adotta uno schema *simil Model-View-Control* che permette una precisa separazione di responsabilità, rendendo il codice il più organizzato e leggibile nel tempo. Di seguito sono riportate le principali "sezioni" in cui il codice sorgente è stato suddiviso:

- **Model:** Sono qui definiti gli oggetti del dominio (es. Chef, Corsi, sessioni).
- **DAO:** Acronimo di *"Data Access Object"* si occupa dell'integrazione e astrazione del database PostgreSQL all'interno dell'applicativo.
- **Controller:** Si occupa di elaborare i comandi dell'utente attraverso la View e ha il compito di eseguire le operazioni utili a modificare lo stato della View stessa.
- **View:** Si occupa di creare una visualizzazione dei dati che sia adatta all'utente finale e ben distante dallo schema fisico del database.

1.2 Model


Notifica
- id_notifica: Integer
- username_chef: String
- titolo: String
- messaggio: String
- solo_iscritti: boolean
- data_creazione: LocalDate
- corso_id: Integer
+ Notifica(Integer id_notifica, String username_chef, String titolo, Integer corso_id)
+ Notifica(String username_chef, String titolo, String messaggio, boolean solo_iscritti, Integer corso_id)
+ getIdNotifica(): Integer
+ getUsernameChef(): String
+ getTitolo(): String
+ getMessaggio(): String
+ isSoloIscritti(): boolean
+ getDataCreazione(): LocalDate
+ getCorsoId(): Integer
+ getUsernameRicevent(): String
+ recMessaggio(): void
+ recSoloIscritti(): void
+ recDataCreazione(): void
+ insert(): void
+ delete(): void
Utente
Abstract
~ username: String
~ password: String
~ nome: String
~ cognome: String
+ Utente(String username, String password)
+ Utente(String username, String password, String nome, String cognome)
+ equals(Object obj): boolean
+ getUsernome():String
+ getPassword():String
+ getNome():String
+ getCognome():String
+ recNome():void
+ recCognome():void
Ingrediente
- nome: String
- unit_misura: String
- quantità: double
+ Ingrediente(String nome, String unit_misura, Double quantità)
+ getNome(): String
+ getUnitMisura(): String
+ getQuantità(): Double
<<Stringa>>
unit_misura
"Kg"
"L"
"Pz"
Ricetta
- nome: String
- descrizione: String
- ingredienti: List<Ingrediente>
+ Ricetta(String nome, String descrizione)
+ equals(Object obj):Boolean
+ getNome(): String
+ getDescrizione(): String
+ getIngredienti(): Array<Ingrediente>
+ getIdRicetta(): Integer
+ recIngredienti(): void
Corso
- id: Integer
- nome: String
- categoria: String
- data_in: LocalDate
- data_fin: LocalDate
- frequenza_settimanale: String
- chef: Chef
+ Corso(Integer id,)
+ Corso(Integer id, String nome, String categoria, LocalDate data_in, LocalDate data_fin, String frequenza_settimanale)
+ Corso(String nome, String categoria, LocalDate data_in, LocalDate data_fin, String frequenza_settimanale chefChef)
+ getId(): integer
+ getNome(): String
+ getNomeById(): String
+ getCategoria(): String
+ getDataIn(): LocalDate
+ getDataFin(): LocalDate
+ getFrequenzaSettimanale(): String
+ getChef(): Chef
+ getSessioniPratiche(): List<Pratica>
+ getSessioniOnline(): List<Online>
+ getStudentiIscritti(): String
+ recChef(): void
+ insert(): boolean
Chef
-resoconto: Report
+ Chef(String username, String password)
+ Chef(String username, String password,String nome, String cognome)
+ getResoconto(): Report
+ getNotifiche():List<Notifica>
+ getCorsi(boolean miei_corsi, String filtro_categoria):ArrayList<Corso>
+ recResoconto(): void
+ verify():boolean
Report
- username_chef: String
- numero_corsi_totali: int
- numero_sessioni_online: int
- numero_sessioni_pratiche: int
- numero_ricette_per_pratiche: Map<Pratica,Integer>
- media_ricette: double
- max_ricette: int
- min_ricette: int
+ Report(String username_chef, int numero_corsi_totali, int numero_sessioni_online, int numero_sessioni_pratiche)
+ getUsernameChef() String
+ getNumeroCorsiTotali(): int
+ getNumeroSessioniOnline() int
+ getNumeroSessioniPratiche(): int
+ recNumeroRicetteParPratiche():void
+ getNumeroRicetteParPratiche():Map<Pratiche, Integer>
+ calcolaStatisticheRicettePratiche(): void
Pratica
- aula: String
- post_totali: int
+ Pratica(Integer corsoId, LocalDate giorno_sessione, String aula, int post_totali)
+ getIdSessione(): Integer
+ getAula(): String
+ getPostTotali(): int
+ getRicette(): ArrayList<Ricetta>
+ getAllRicette(): ArrayList<Ricetta>
+ getNStudentiIscritti(): int
+ getStudentiIscritti(): String
+ getIngredientiPratica(): List<Ingrediente>
+ insert(): boolean
+ update(Online online): boolean
+ delete(): boolean
+ aggiungiRicetta(Ricetta ricetta) : boolean
Online
- codice_meeting: String
+ Online(Integer corsoId, LocalDate giorno_sessione, String codice_meeting)
+ getIdSessione() Integer
+ getCodiceMeeting() String
+ insert(): boolean
+ update(Online online): boolean
+ delete(): boolean
Sessione
Abstract
~ corso_id: Integer
~ giorno_sessione: LocalDate
+ Sessione(Integer corso_id, LocalDate giorno_sessione)
+ equals(Object obj): boolean
+ getCorsoId(): integer
+ getGiornoSessione(): LocalDate
+ [Abstract] getIdSessione(): Integer
Creato_da
Frequenta_da
Frequenta_da
Pertinenza
1
1
1

1.2.1 Descrizione delle entità:

- **Utente:** *Classe astratta* che rappresenta un utente generico con username, password, nome e cognome.
- **Chef:** *Estende Utente* e rappresenta uno chef che gestisce corsi, riceve notifiche e ottiene e scarica un resoconto statistico.
- **Corso:** Rappresenta un corso di cucina con nome, categoria, date, frequenza settimanale e chef responsabile.
- **Ricetta:** Ricetta con nome, descrizione e lista di ingredienti.
- **Ingrediente:** Ingrediente con nome, unità di misura e quantità.
- **Sessione:** *Classe astratta* che rappresenta una sessione generica, contiene data e corsoid.
- **Pratica:** Estende Sessione. Rappresenta una sessione pratica in aula con numero di posti e ricette associate.
- **Online:** *Estende Sessione* e rappresenta una sessione online con codice del meeting.
- **Notifica:** Rappresenta una notifica/annuncio inviata da uno chef ai suoi studenti, con titolo, messaggio e data di creazione.
- **Report:** Representa un resoconto (consultabile come numerico e grafico) per uno chef con dati su corsi, sessioni e ricette.

1.3 DAO

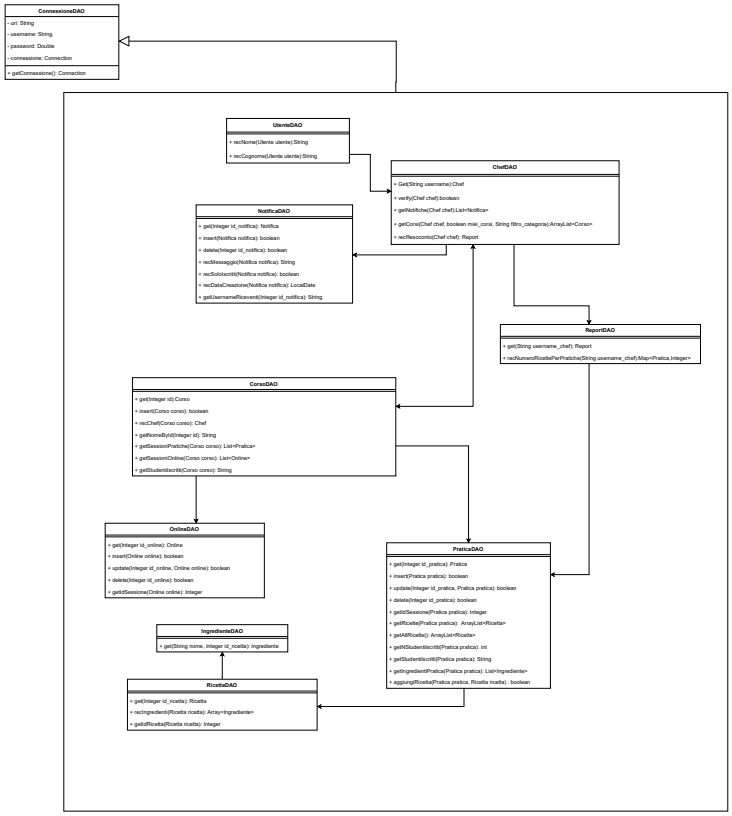


Figura 2: Schema del *Data Access Object*

1.3.1 Riassunto e classi DAO

- **ConnessioneDAO:** Fornisce la connessione con il DB PostgreSQL
- **UtenteDAO:** Recupera i dati anagrafici base di un utente a partire dallo username.
- **ChefDAO:** Gestisce operazioni legate allo chef: recupero chef, verifica credenziali chef, elenco notifiche, elenco corsi (miei/altri con filtro per categoria), recupero report.
- **CorsoDAO:** Gestisce il recupero dei corsi, nome, studenti iscritti, sessioni pratiche/online, chef associato e inserimento di un nuovo corso.
- **PraticaDAO:** Gestisce le sessioni pratiche, occupandosi di recupero, iscritti, ingredienti e ricette, elenco ricette, funzioni insert/update/delete e associazione ricetta-to-pratica.
- **OnlineDAO:** Gestisce le sessioni online: recupero sessione specifica e gestione insert/update/delete.
- **NotificaDAO:** Gestisce le notifiche: recupero notifica e aggiunta, destinatari, messaggio, solo iscritti, data
- **RicettaDAO:** Gestisce ricette e ingredienti collegati: recupero ricetta, recupero ingredienti tramite tabella di relazione.
- **IngredienteDAO:** Recupera un ingrediente con unità di misura e quantità legato ad una ricetta precisa.
- **ReportDAO:** Recupera il report messo a punto per lo chef.

Nota: Per l'implementazione delle funzioni base sui dati, è stato fatto uso di procedure CRUD (Create,Read,Update,Delete) attraverso query SQL.

1.4 Controller

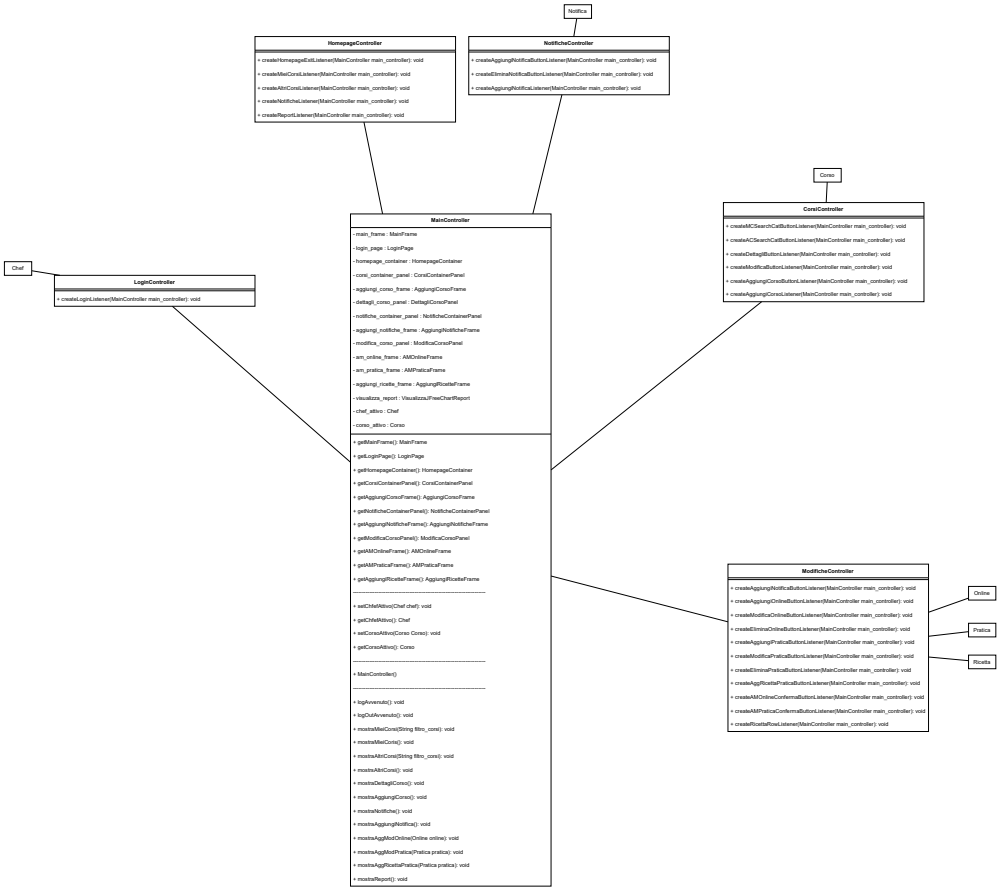


Figura 3: Schema del Controller

1.4.1 Riassunto e classi Controller

1. **HomepageController:** Gestisce la navigazione partendo dalla homepage: logout, apertura *"miei corsi"*, *"altri corsi"*, *"notifiche"* e *"report"*, con ricarica dello stato del corso attivo quando necessario(es. quando avviene uno switch).

2. **LoginController:** Gestisce il processo di login: legge credenziali inserite nella UI, autentica lo chef, effettua l'aggiornamento della interfaccia e gestisce i messaggi di successo/errore dedicati.

3. **CorsiController:** Gestisce le interazioni utente nella sezione dei corsi: ricerca per *"categoria"*, apertura *"dettagli"*/*"modifica"* e form di aggiunta, oltre alla conferma inserimento di nuovi, impedendo inserimenti invalidi.

4. **ModificheController:** Controller della modifica dei corsi e affini: gestione aggiunta/modifica/eliminazione sessioni online e pratiche, apertura finestre, e associazione ricette alle pratiche.

5. **NotificheController:** Gestisce la sezione notifiche: apertura form di aggiunta(sia dedicato a studenti iscritti che globale), eliminazione e inserimento.

6. **Maincontroller:** Coordinatore principale dell'app Swing, che consente il collegamento tra: *View*, permettendo lo spostamento tra schermate; *Controller* specifici e listeners; *Model* e chiamata a *main*.

1.5 View

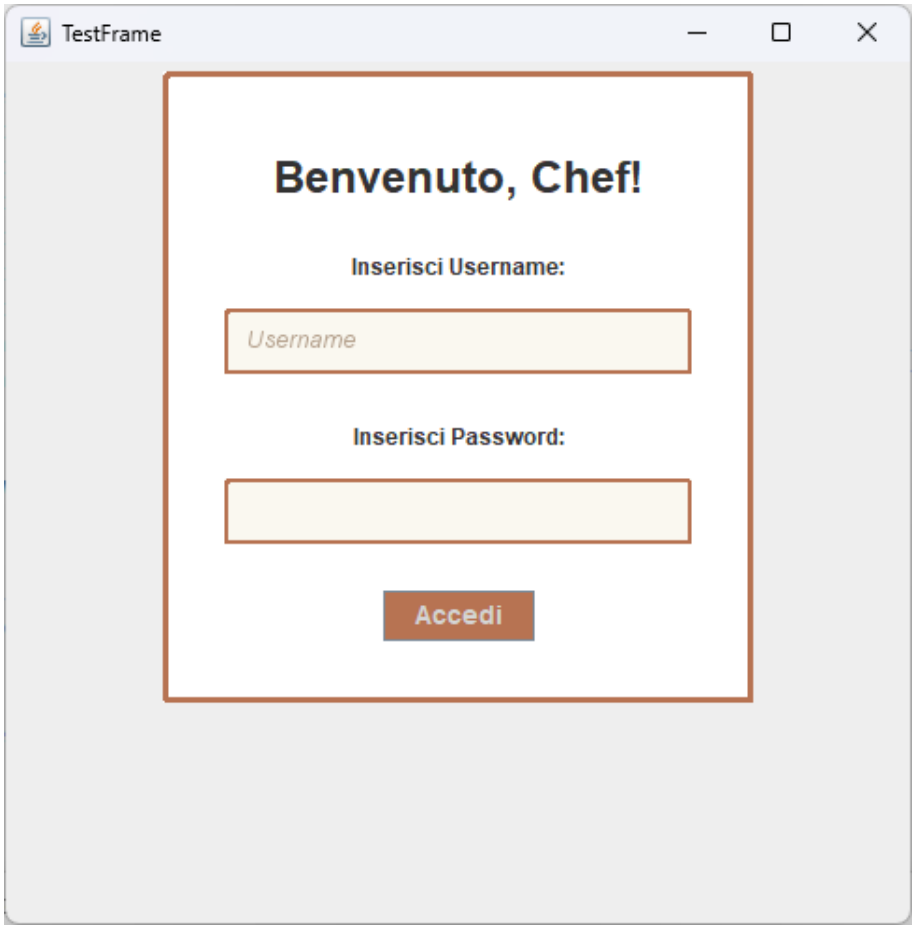


Figura 4: Scheda login Chef

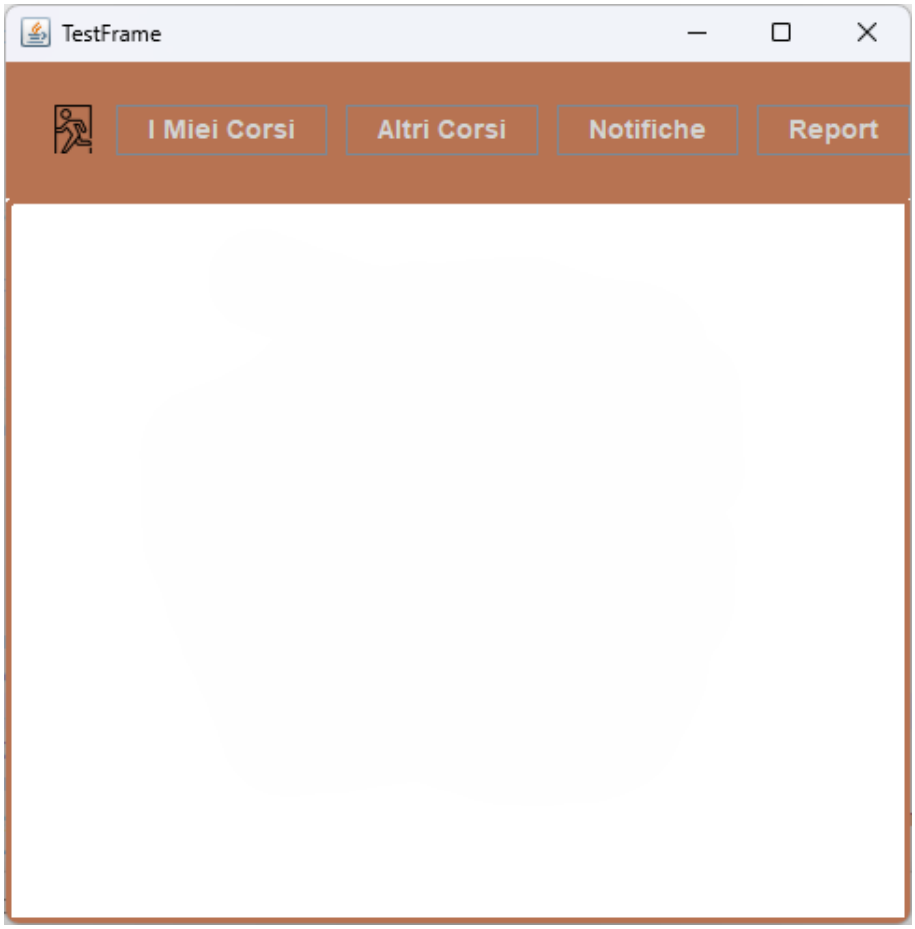


Figura 5: Barra di selezione superiore - Homepage

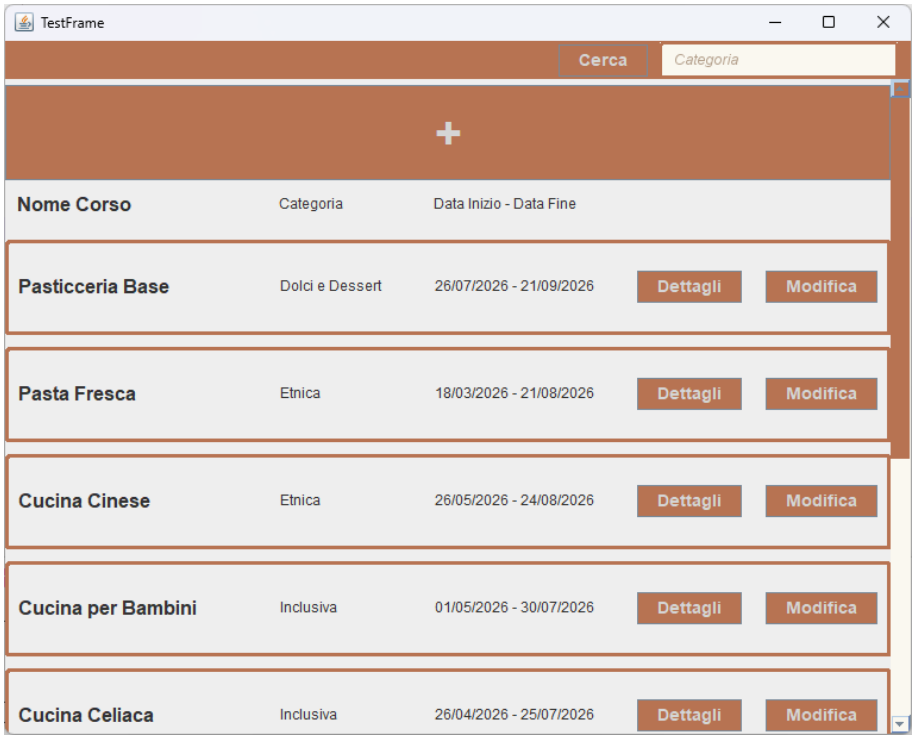


Figura 6: Visualizzazione Corsi e ricerca

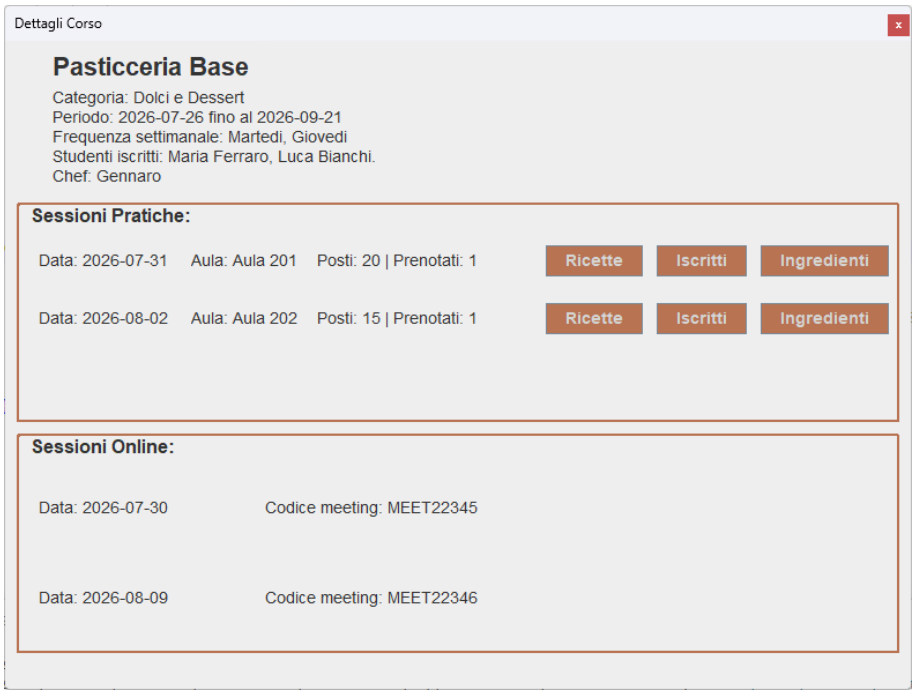


Figura 7: Dettagli Corso e Sessioni

Figura 8: Interfaccia modifica e gestione delle Sessioni

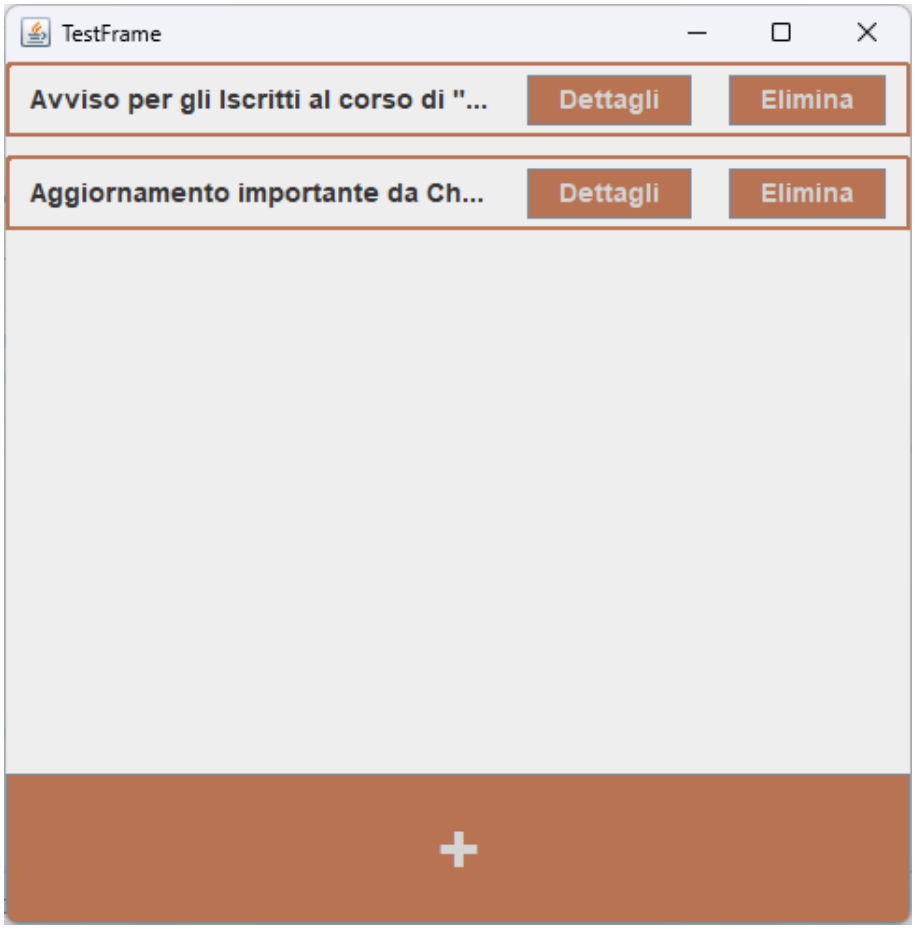


Figura 9: Visualizzazione e aggiunta delle Notifiche

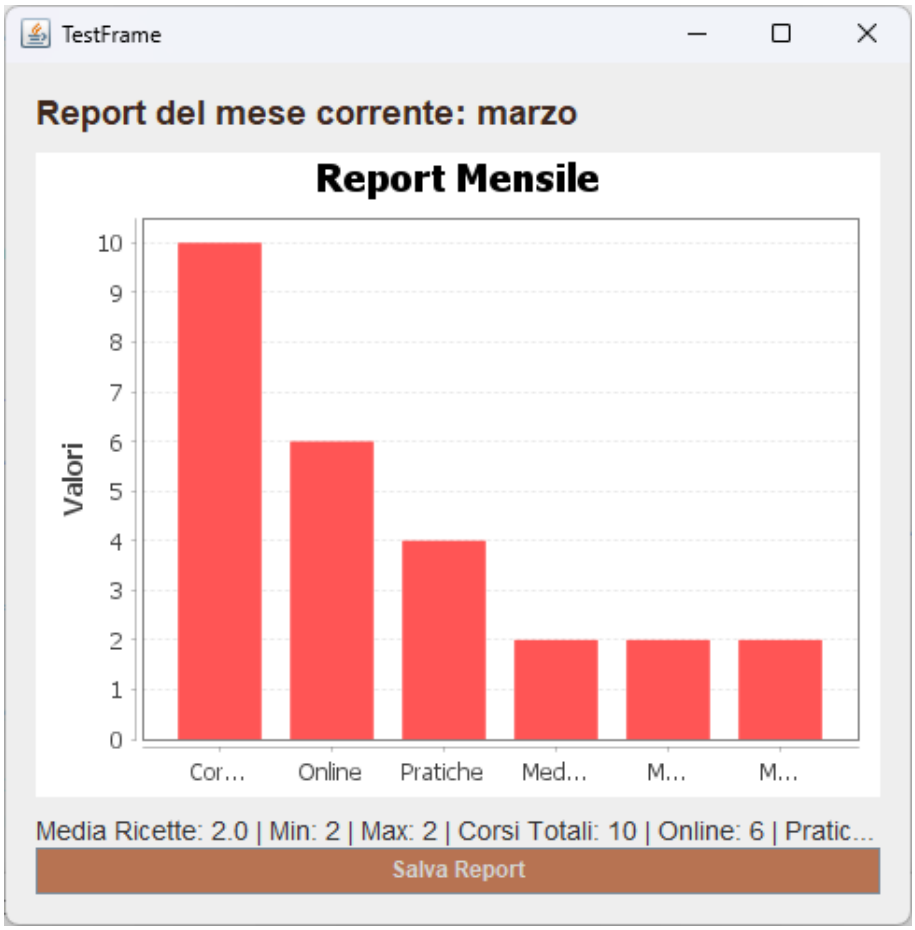


Figura 10: Resoconto grafico personale dello Chef

Note - View:

- Alcuni "glitch" presenti nelle immagini sono dovute alla esecuzione dei container in ambiente isolato.
- Immagini come "*Visualizzazione delle notifiche*" e "*Dettagli Corsi e Sessioni*" sono da intendere valide sia per la consultazione dedicata allo Chef attivo sia per la visualizzazione globale.
- Le Notifiche presentano una doppia interfaccia di aggiunta: una pensata per notifiche dedicate ai soli iscritti ai corsi (*Interfaccia modifica e gestione delle Sessioni*) l'altra per tutti (*Visualizzazione e aggiunta delle Notifiche*).

1.6 GitHub

Clicca qui